

TUGAS PRAKTIKUM 6
“DEADLOCK”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Operasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Nama : Muhammad Fahrur Rohman
NIM : 2410651056
Kelas : Kamis-C

UNIVERSITAS MUHAMMADIYAH JEMBER
FAKULTAS TEKNIK
PRODI TEKNIK INFORMATIKA
2025-2026

TUGAS 1: ANALISIS DEADLOCK

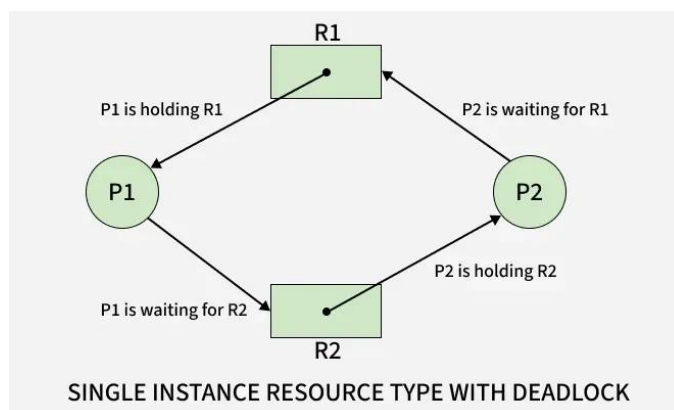
Mengapa program mengalami deadlock?

Program tersebut mengalami deadlock karena terdapat dua thread yang saling menunggu resource yang sedang dipegang oleh thread lain sehingga tidak ada yang bisa melanjutkan prosesnya contohnya di praktikum. Thread 1 memegang lock 1 dan menunggu lock 2 yang dipegang oleh Thread 2 sedangkan Thread 2 memegang lock 2 lalu menunggu lock 1 yang dipegang oleh Thread 1.

Kondisi deadlock apa saja yang terpenuhi?

Kondisi deadlock yang terpenuhi antara lain, yang pertama ada Mutual Exclusion, yaitu kondisi Dimana setiap lock 1 dan lock 2 hanya bisa dipegang oleh satu thread pada satu waktu, kemudian yang kedua ada Hold and Wait, yaitu seperti pada pertanyaan pertama tadi, thread 1 memegang lock 1 sambil menunggu lock 2 sedangkan thread 2 memegang lock 2 sambil menunggu lock 1, kemudian yang ketiga ada No Preemption, artinya thread harus melepas sendiri locknya karena mutex tidak bisa dicabut secara paksa oleh system, dan yang terakhir ada Circular Wait, yaitu terjadi lingkaran saling menunggu tanpa ada yang bisa melanjutkan.

Gambar diagram resource allocation graphnya!



TUGAS 2: IMPLEMENTASI PENCEGAHAN

Jadi pada tugas kedua ini, kita akan membuat pencegahan deadlocknya, namun dengan dua teknik yang berbeda lalu membandingkan keduanya, yaitu Lock Ordering dan Try Lock. Cukup modifikasi program yang deadlock dengan kode berikut ini, masing-masing dengan urutan lock ordering dan try lock, tapi saya disini membuat program baru lagi di direktori tugas, lalu jalankan.

Lock Ordering:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Helper: semua thread ambil lock dengan urutan sama
void acquire_locks_in_order() {
    pthread_mutex_lock(&lock1);
    pthread_mutex_lock(&lock2);
}

void release_locks_in_order() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}

void* thread1_func(void* arg) {
    printf("Thread 1: mulai\n");

    acquire_locks_in_order();
    printf("Thread 1: sedang menggunakan resource\n");
    sleep(1);
    release_locks_in_order();

    printf("Thread 1: selesai\n");
    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: mulai\n");

    acquire_locks_in_order();
    printf("Thread 2: sedang menggunakan resource\n");
    sleep(1);
    release_locks_in_order();

    printf("Thread 2: selesai\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n=== SELESAI TANPA DEADLOCK ===\n");
    return 0;
}
```

Penjelasan:

acquire_locks_in_order()

berfungsi mengatur urutan agar kedua thread selalu mengambil lock1 terlebih dahulu dan kemudian lock2 sehingga tidak terjadi proses saling tunggu. Kedua thread memanggil ini sebelum bekerja.

release_locks_in_order()

Berfungsi untuk melepas lock 2 terlebih dahulu, baru kemudian lock 1 untuk memastikan kedua resource Kembali dengan benar.

Try Lock:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_func(void* arg) {
    int retry = 0;
    while (1) {
        pthread_mutex_lock(&lock1);

        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1: dapat kedua lock\n");

            sleep(1);

            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        } else {
            printf("Thread 1: gagal lock2, retry...\n");
            pthread_mutex_unlock(&lock1);
            retry++;
            usleep(100000); // 0.1 detik
        }
    }
    printf("Thread 1 selesai (retry: %d)\n", retry);
    return NULL;
}

void* thread2_func(void* arg) {
    int retry = 0;
    while (1) {
        pthread_mutex_lock(&lock2);

        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2: dapat kedua lock\n");

            sleep(1);

            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        } else {
            printf("Thread 2: gagal lock1, retry...\n");
            pthread_mutex_unlock(&lock2);
            retry++;
            usleep(150000); // 0.15 detik
        }
    }
    printf("Thread 2 selesai (retry: %d)\n", retry);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: TRY-LOCK ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n=== SELESAI TANPA DEADLOCK ===\n");
    return 0;
}
```

Penjelasan:

pthread_mutex_trylock()

Fungsi try-lock tidak memungkinkan thread menunggu apabila lock dipegang oleh thread lain. Jika lock tidak tersedia, fungsi ini langsung gagal dan mengembalikan nilai non-zero. Oleh karena itu, thread tidak pernah berada pada kondisi "menunggu selamanya", yang merupakan salah satu syarat terjadinya deadlock. Ketika try-lock gagal, thread langsung melepaskan lock pertama yang ia pegang dengan menggunakan `pthread_mutex_unlock(&lock1)` atau `pthread_mutex_unlock(&lock2)`, lalu melakukan percobaan ulang setelah menunggu sebentar. Dalam proses ini, kondisi hold-and-wait dihilangkan karena tidak ada thread yang memegang lock satu sambil men Thread hanya memegang kunci ketika kedua kunci tersedia. Meskipun loop retry yang menggunakan `usleep()` tidak mencegah deadlock secara langsung, itu memberi waktu kepada thread lain untuk menyelesaikan tugasnya dan melepaskan lock.

Hasil:

```
fahru@FAHRUR:~/praktikum6/tugas$ nano lock_ordering.c
fahru@FAHRUR:~/praktikum6/tugas$ gcc lock_ordering.c -o lock_ordering
fahru@FAHRUR:~/praktikum6/tugas$ ./
-bash: ./: Is a directory
fahru@FAHRUR:~/praktikum6/tugas$ ./lock_ordering
=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===

Thread 1: mulai
Thread 1: sedang menggunakan resource
Thread 2: mulai
Thread 1: selesai
Thread 2: sedang menggunakan resource
Thread 2: selesai

=== SELESAI TANPA DEADLOCK ===
fahru@FAHRUR:~/praktikum6/tugas$ nano try_lock.c
fahru@FAHRUR:~/praktikum6/tugas$ gcc try_lock.c -o try_lock
fahru@FAHRUR:~/praktikum6/tugas$ ./try_lock
=== PENCEGAHAN DEADLOCK: TRY-LOCK ===

Thread 1: dapat kedua lock
Thread 2: dapat kedua lock
Thread 1 selesai (retry: 0)
Thread 2 selesai (retry: 0)

=== SELESAI TANPA DEADLOCK ===
fahru@FAHRUR:~/praktikum6/tugas$ |
```

Bisa kita lihat pada hasil diatas, keduanya sama-sama berhasil mencegah deadlock, namun dengan cara yang berbeda. Pada Lock Ordering, program ini bekerja dengan cara memastikan semua thread mengambil resource dalam urutan yang sama sehingga memicu proses saling menunggu, sedangkan pada Try Lock, program bekerja dengan cara jika semisal thread mencoba mengambil lock kedua namun lock kedua sedang dipakai, thread akan melepas lock yang telah ia pegang lalu mencoba lagi nanti sehingga tidak ada thread yang memegang satu lock sambil menunggu lock lain yang mencegah kondisi hold and wait terjadi.

TUGAS 3: BANKER'S ALGORITHM

Disini, kita diperintahkan membuat program dengan ketentuan memiliki 4 proses, 3 jenis resource, available diinisialisasi sebagai {5, 4, 3}, serta mengimplementasi request resource dan cek keamanan sistem. Pertama-tama, disini saya membuat program bernama dengan isi kodenya sebagai berikut, lalu jalankan dan hasilnya seperti pada gambar dibawah kode.

Kode:

```
#include <stdio.h>
#include <stdbool.h>

#define P 5 // jumlah proses
#define R 3 // jumlah jenis resource

// Data sistem
int available[R] = {3, 3, 2}; // Resource yang tersedia

int maximum[P][R] = { // Kebutuhan maksimum setiap proses
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2},
    {4, 3, 3}
};

int allocation[P][R] = { // Resource yang sudah dialokasikan
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1},
    {0, 0, 2}
};

int need[P][R]; // Kebutuhan yang tersisa

// Fungsi untuk menghitung kebutuhan
void calculate_need() {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}

// Fungsi untuk mencetak status sistem
void print_status() {
    printf("\n=== STATUS SISTEM ===\n");
    printf("\nAvailable: ");
    for (int i = 0; i < R; i++) printf("%d ", available[i]);
    printf("\n\nAllocation:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", allocation[i][j]);
        printf("\n");
    }
    printf("\nNeed:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
}
```

```

// Fungsi untuk mengecek apakah sistem dalam keadaan aman
bool is_safe() {
    int work[R];
    bool finish[P] = {false};
    int safe_sequence[P];
    int count = 0;

    // Copy available ke work
    for (int i = 0; i < R; i++) {
        work[i] = available[i];
    }

    printf("\n=== MEMERIKSA KEAMANAN SISTEM ===\n");

    // Cari urutan aman
    while (count < P) {
        bool found = false;

        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool can_allocate = true;

                // Cek apakah need[i] <= work
                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        can_allocate = false;
                        break;
                    }
                }

                if (can_allocate) {
                    printf("P%d dapat dijalankan. ", i);
                    printf("Work: [");
                    for (int j = 0; j < R; j++) printf("%d ", work[j]);
                    printf("]\n");

                    // Simulasi proses selesai
                    for (int j = 0; j < R; j++) {
                        work[j] += allocation[i][j];
                    }

                    safe_sequence[count++] = i;
                    finish[i] = true;

                    found = true;
                }
            }
        }

        if (!found) {
            printf("\n[PERINGATAN] Tidak dapat menemukan urutan aman!\n");
            return false;
        }
    }
}

```

```

printf("\n[SUKSES] Sistem dalam keadaan AMAN!\n");
printf("Safe Sequence: ");
for (int i = 0; i < P; i++) {
    printf("%d ", safe_sequence[i]);
    if (i < P - 1) printf(" ");
}
printf("\n");

return true;
}

// Fungsi untuk meminta resource
bool request_resources(int process, int request[]) {
    printf("\n=== REQUEST DARI %d ===\n", process);
    printf("Meminta: ");
    for (int i = 0; i < R; i++) printf("%d ", request[i]);
    printf("\n");

    // Cek apakah request <= need
    for (int i = 0; i < R; i++) {
        if (request[i] > need[process][i]) {
            printf("[DITOLAK] Request melebihi kebutuhan maksimum!\n");
            return false;
        }
    }

    // Cek apakah request <= available
    for (int i = 0; i < R; i++) {
        if (request[i] > available[i]) {
            printf("[DITOLAK] Resource tidak tersedia!\n");
            return false;
        }
    }

    // Simulasi alokasi
    for (int i = 0; i < R; i++) {
        available[i] -= request[i];
        allocation[process][i] += request[i];
        need[process][i] -= request[i];
    }

    // Cek keamanan
    if (is_safe()) {
        printf("[DITERIMA] Request berhasil dialokasikan!\n");
        return true;
    } else {
        // Rollback
        printf("[DITOLAK] Request menyebabkan unsafe state!\n");
        for (int i = 0; i < R; i++) {
            available[i] += request[i];
            allocation[process][i] -= request[i];
            need[process][i] += request[i];
        }
        return false;
    }
}

```

```

int main() {
    printf("=== BANKER'S ALGORITHM ===\n");

    calculate_need();
    print_status();

    // Cek keadaan awal
    is_safe();

    // Simulasi request 1 (aman)
    int request1[R] = {1, 0, 2};
    request_resources(1, request1);
    print_status();

    // Simulasi request 2 (tidak aman)
    int request2[R] = {3, 3, 0};
    request_resources(4, request2);

    printf("\n=== PROGRAM SELESAI ===\n");
    return 0;
}

```

Hasil:

```
fahru@FAHRUR:~/praktikum6$ nano bankers_algorithm.c
fahru@FAHRUR:~/praktikum6$ gcc bankers_algorithm.c -o bankers_algorithm
fahru@FAHRUR:~/praktikum6$ ./bankers_algorithm
=== BANKER'S ALGORITHM ===

=== STATUS SISTEM ===

Available: 3 3 2

Allocation:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

Need:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

=== MEMERIKSA KEAMANAN SISTEM ===
P1 dapat dijalankan. Work: [3 3 2]
P3 dapat dijalankan. Work: [5 3 2]
P4 dapat dijalankan. Work: [7 4 3]
P0 dapat dijalankan. Work: [7 4 5]
P2 dapat dijalankan. Work: [7 5 5]

[SUKSES] Sistem dalam keadaan AMAN!
Safe Sequence: P1 → P3 → P4 → P0 → P2

=== REQUEST DARI P1 ===
Meminta: 1 0 2

=== MEMERIKSA KEAMANAN SISTEM ===
P1 dapat dijalankan. Work: [2 3 0]
P3 dapat dijalankan. Work: [5 3 2]
P4 dapat dijalankan. Work: [7 4 3]
P0 dapat dijalankan. Work: [7 4 5]
P2 dapat dijalankan. Work: [7 5 5]

[SUKSES] Sistem dalam keadaan AMAN!
Safe Sequence: P1 → P3 → P4 → P0 → P2
[DITERIMA] Request berhasil dialokasikan!

=== STATUS SISTEM ===

Available: 2 3 0

Allocation:
P0: 0 1 0
P1: 3 0 2
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

Need:
P0: 7 4 3
P1: 0 2 0
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

=== REQUEST DARI P4 ===
Meminta: 3 3 0
[DITOLAK] Resource tidak tersedia!

=== PROGRAM SELESAI ===
```

Berdasarkan hasilnya pada gambar diatas, bis akita lihat bahwa sistem awal berada dalam kondisi aman dibuktikan dengan adanya *safe sequence* yang berhasil ditemukan, yaitu urutan eksekusi proses $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$. Urutan ini menunjukkan bahwa seluruh proses dapat dijalankan secara berurutan tanpa menyebabkan deadlock, karena setiap proses memperoleh jumlah resource yang diperlukan setelah proses sebelumnya selesai dan mengembalikan resource miliknya. Pada akhir, permintaan ditolak karena resource yang diminta melebihi resource yang tersedia.

TUGAS 4: KASUS NYATA

Disini kita diperintahkan membuat simulasi suatu kasus dimana dua orang ingin transfer uang antar rekening secara bersamaan dengan ketentuan orang A transfer dari rekening 1 ke rekening 2 sedangkan orang B transfer dari rekening 2 ke rekening 1. Disini saya menggunakan teknik lock ordering. Pertama-tama, buat program deadlocknya terlebih dahulu kemudian buat program preventionnya dengan isi kode masing-masing secara berurutan sebagai berikut, kemudian jalankan.

```
/* deadlock_transfer.c
   Simulasi deadlock: dua thread transfer uang antar 2 rekening
   Compile: gcc deadlock_transfer.c -o deadlock_transfer -pthread
   Jalankan: ./deadlock_transfer (program akan hang karena deadlock)
*/

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

typedef struct {
    int id;
    int balance;
    pthread_mutex_t lock;
} Account;

Account acc1 = { .id = 1, .balance = 100, .lock = PTHREAD_MUTEX_INITIALIZER };
Account acc2 = { .id = 2, .balance = 100, .lock = PTHREAD_MUTEX_INITIALIZER };

void transfer(Account *from, Account *to, int amount) {
    // ambil lock from dulu, lalu lock to (urutan berbeda di thread lain ->
    deadlock)
    printf("Thread %ld: mencoba lock account %d\n", pthread_self(), from->id);
    pthread_mutex_lock(&from->lock);
    printf("Thread %ld: terkunci account %d\n", pthread_self(), from->id);

    sleep(1); // beri waktu agar thread lain mengambil lock lain -> memicu
    deadlock
```

```

printf("Thread %ld: mencoba lock account %d\n", pthread_self(), to->id);
pthread_mutex_lock(&to->lock);
printf("Thread %ld: terkunci account %d\n", pthread_self(), to->id);

// lakukan transfer
if (from->balance >= amount) {
    from->balance -= amount;
    to->balance += amount;
    printf("Thread %ld: transfer %d dari %d ke %d berhasil\n",
        pthread_self(), amount, from->id, to->id);
} else {
    printf("Thread %ld: saldo tidak cukup\n", pthread_self());
}

pthread_mutex_unlock(&to->lock);
pthread_mutex_unlock(&from->lock);
}

void* personA(void* arg) {
    // A: transfer dari acc1 ke acc2
    transfer(&acc1, &acc2, 10);
    return NULL;
}

void* personB(void* arg) {
    // B: transfer dari acc2 ke acc1
    transfer(&acc2, &acc1, 20);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("Saldo awal: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);

    pthread_create(&t1, NULL, personA, NULL);
    pthread_create(&t2, NULL, personB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Saldo akhir: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);
    return 0;
}

```

Prevention:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

typedef struct {
    int id;
    int balance;
    pthread_mutex_t lock;
} Account;

Account acc1 = { .id = 1, .balance = 100, .lock = PTHREAD_MUTEX_INITIALIZER };
Account acc2 = { .id = 2, .balance = 100, .lock = PTHREAD_MUTEX_INITIALIZER };

void transfer_ordered(Account *from, Account *to, int amount) {
    Account *first = (from->id < to->id) ? from : to;
    Account *second = (from->id < to->id) ? to : from;

    // ambil lock berdasarkan urutan id (deterministik)
    pthread_mutex_lock(&first->lock);
    pthread_mutex_lock(&second->lock);

    // lakukan transfer (pastikan still consistent)
    if (from->balance >= amount) {
        from->balance -= amount;
        to->balance += amount;
        printf("Transfer %d dari %d ke %d berhasil\n", amount, from->id, to->id);
    } else {
        printf("Saldo tidak cukup untuk transfer %d dari %d ke %d\n", amount, from->id, to->id);
    }

    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
}

void* personA(void* arg) {
    transfer_ordered(&acc1, &acc2, 10);
    return NULL;
}

void* personB(void* arg) {
    transfer_ordered(&acc2, &acc1, 20);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    printf("Saldo awal: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);

    pthread_create(&t1, NULL, personA, NULL);
    pthread_create(&t2, NULL, personB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Saldo akhir: acc1=%d, acc2=%d\n", acc1.balance, acc2.balance);
    return 0;
}
```

Hasil:

```
fahru@FAHRUR:~/praktikum6/tugas$ nano transfer_deadlock.c
fahru@FAHRUR:~/praktikum6/tugas$ gcc transfer_deadlock.c -o transfer_deadlock
fahru@FAHRUR:~/praktikum6/tugas$ ./transfer_deadlock
Saldo awal: acc1=100, acc2=100
Thread 129285675284032: mencoba lock account 1
Thread 129285675284032: terkunci account 1
Thread 129285666891328: mencoba lock account 2
Thread 129285666891328: terkunci account 2
Thread 129285675284032: mencoba lock account 2
Thread 129285666891328: mencoba lock account 1
^C
fahru@FAHRUR:~/praktikum6/tugas$ nano transfer_prevention.c
fahru@FAHRUR:~/praktikum6/tugas$ gcc transfer_prevention.c -o transfer_prevention
fahru@FAHRUR:~/praktikum6/tugas$ ./transfer_prevention
Saldo awal: acc1=100, acc2=100
Transfer 10 dari 1 ke 2 berhasil
Transfer 20 dari 2 ke 1 berhasil
Saldo akhir: acc1=110, acc2=90
fahru@FAHRUR:~/praktikum6/tugas$ |
```

Dari gambar diatas, bisa kita lihat bahwa deadlock diawal bisa teratasi diakhirnya menggunakan Teknik lock ordering, yaitu mengunci (lock) rekening berdasarkan urutan yang konsisten yang membuat kedua thread mengambil lock dalam urutan yang sama sehingga proses saling menunggu tidak terjadi dan deadlock tercegah, yaitu transfer 10 dari rekening akun orang A (acc1) ke rekening akun orang B (acc2) dan transfer 20 dari rekening akun orang B (acc2) ke rekening akun orang A (acc1) berhasil.